1. m09 Boosting

Boosting: iteratively add basis functions (typically shallow trees) in a greedy fashion so each addition reduces the chosen loss. Applied to models with high bias / low variance (esp. trees). Final model is an additive expansion

$$f(x) = \sum_{m=1}^{M} \beta_m\, b(x;\gamma_m), \qquad f_M(x) = \sum_{m=1}^{M} T(x;\Theta_m).$$

1.1 Three main ingredients

Ingredient	Role
Weak learner	High-bias / low-variance base (typ. shallow tree / stump).
Loss L	Differentiable cost we want to minimise.
Additive model	Sequentially add weak learners, each lowering L.

1.2 AdaBoost.M1 (Algorithm 10.1, ESL)

Binary classification, $Y \in \{-1, +1\}$, weak classifier $G(x)$.

Algorithm 1 AdaBoost.M1 (ESL Alg. 10.1)

1: Initialise weights $w_i = 1/N$, $i = 1, \dots, N$.
2: **for** $m = 1, 2, \dots, M$ **do**
3: Fit $G_m(x)$ to training data using weights w_i.
4: $\mathrm{err}_m \leftarrow \dfrac{\sum_{i=1}^{N} w_i\, \mathbb{I}(y_i \neq G_m(x_i))}{\sum_{i=1}^{N} w_i}$
5: $\alpha_m \leftarrow \log\big((1 - \mathrm{err}_m)/\mathrm{err}_m\big)$
6: $w_i \leftarrow w_i \cdot \exp\big(\alpha_m\, \mathbb{I}(y_i \neq G_m(x_i))\big)$, $i = 1, \dots, N$
7: **end for**
8: **return** $G(x) = \mathrm{sign}\Big[\sum_{m=1}^{M} \alpha_m G_m(x)\Big]$

Notes. $\alpha_m > 0$ iff $\mathrm{err}_m < 0.5$; misclassified points get multiplied by $\exp(\alpha_m)$, correctly classified ones keep their weight; output is the sign of a continuous score. AdaBoost equals forward-stagewise additive modelling with the exponential loss $L(y, f) = \exp(-yf)$ (discovered 5 yrs after invention). (See ESL Fig. 10.1 for the schematic.)

1.3 Boosting regression trees (ISLP Alg. 8.2)

Algorithm 2 Boosting for regression trees (ISLP Alg. 8.2)

1: Set $\hat{f}(x) = 0$ and $r_i = y_i$ for all i in training set.
2: **for** $b = 1, 2, \dots, B$ **do**
3: Fit tree $\hat{f}^b$ with d splits ($d+1$ terminal nodes) to data (X, r).
4: Update $\hat{f}(x) \leftarrow \hat{f}(x) + \nu\, \hat{f}^b(x)$
5: Update residuals $r_i \leftarrow r_i - \nu\, \hat{f}^b(x_i)$
6: **end for**
7: **return** $\hat{f}(x) = \sum_{b=1}^{B} \nu\, \hat{f}^b(x)$

Three tunings: B (number of trees), ν (shrinkage), d (interaction depth). $d = 1$ gives an additive model.

1.4 Forward-stagewise additive modelling (boosting trees)

Single tree as $T(x;\Theta) = \sum_{j=1}^{J} \gamma_j\, \mathbb{I}(x \in R_j)$, parameters $\Theta = \{R_j, \gamma_j\}_1^J$. Boosting solves at step m:

$$\hat{\Theta}_m = \arg\min_{\Theta_m} \sum_{i=1}^{N} L\big(y_i,\, f_{m-1}(x_i) + T(x_i;\Theta_m)\big).$$

Two sub-problems: (i) given R_j, find γ_j (easy: mean / majority vote); (ii) find R_j (hard, greedy / surrogate loss).

1.5 Steepest descent in function space

Loss $L(f) = \sum_i L(y_i, f(x_i))$ has gradient vector $g_m^\top = (g_{1m}, \dots, g_{Nm})$ with components

$$g_{im} = \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f = f_{m-1}}.$$

Update via line-search step $\rho_m = \arg\min_\rho L(f_{m-1} - \rho g_m)$, then $f_m = f_{m-1} - \rho_m g_m$. (See ESL Fig. for steepest descent; `graphics/Steepest_descet.png`.)

1.6 Gradient tree boosting (ESL Alg. 10.3)

Central idea: fit a tree as close as possible to the negative gradient:

$$\tilde{\Theta}_m = \arg\min_{\Theta} \sum_{i=1}^{N} (-g_{im} - T(x_i;\Theta))^2.$$

That is: fit tree to negative gradient by least squares; then optimise γ_{jm} on the resulting regions.

Algorithm 3 Gradient tree boosting (ESL Alg. 10.3)

1: Initialise $f_0(x) = \arg\min_\gamma \sum_{i=1}^{N} L(y_i, \gamma)$.
2: **for** $m = 1$ to M **do**
3: For $i = 1, \dots, N$ compute pseudo-residuals

$$r_{im} = -\left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f = f_{m-1}}$$

4: Fit regression tree to targets r_{im} giving regions R_{jm}, $j = 1, \dots, J_m$.
5: For $j = 1, \dots, J_m$ compute

$$\gamma_{jm} = \arg\min_{\gamma} \sum_{x_i \in R_{jm}} L\big(y_i, f_{m-1}(x_i) + \gamma\big)$$

6: Update $f_m(x) = f_{m-1}(x) + \sum_{j=1}^{J_m} \gamma_{jm}\, \mathbb{I}(x \in R_{jm})$
7: **end for**
8: **return** $\hat{f}(x) = f_M(x)$

(See ESL Fig. 10.3 `graphics/Algorithm10.3.png`.) The r_{im} are called generalised / pseudo-residuals.

1.7 Loss / objective table

Loss	$L(y, f)$	$-g_i = -\partial L/\partial f$
Quadratic	$\frac{1}{2}(y - f)^2$	$y - f$ (residual)
Absolute	$\lvert y - f\rvert$	$\mathrm{sign}(y - f)$
Huber	$\begin{cases} (y-f)^2 & \lvert y-f\rvert \le \delta \\ 2\delta\lvert y-f\rvert - \delta^2 & \text{else} \end{cases}$	piecewise
Exponential (AdaBoost)	$\exp(-yf)$, $y \in \{-1, +1\}$	$y \exp(-yf)$
Binomial deviance	$-\mathbb{I}(y{=}1)\log p - \mathbb{I}(y{=}0)\log(1-p)$	$y - p(x)$
Multinomial deviance	$-\sum_{k=1}^{K} \mathbb{I}(y{=}k) \log p_k(x)$	$\mathbb{I}(y_i{=}k) - p_k(x_i)$

For binomial deviance: $p(x) = e^{f(x)}/(1 + e^{f(x)})$ (logistic / sigmoid), $y \in \{0, 1\}$.

For multinomial deviance: $p_k(x) = e^{f_k(x)} / \sum_{l=1}^{K} e^{f_l(x)}$ (softmax). Builds K trees per boosting round, one per class, each fit to g_{km}; aggregate class probabilities in step 3.

(See ESL Fig. for robust losses, `graphics/robust_loss.png`.)

1.8 Shrinkage / learning-rate update

Scale each new tree's contribution by $\nu \in (0, 1]$:

$$f_m(x) = f_{m-1}(x) + \nu \sum_{j=1}^{J_m} \gamma_{jm}\, \mathbb{I}(x \in R_{jm}).$$

ν and M are linked: smaller $\nu \Rightarrow$ larger M. Empirically $\nu < 0.1$ is more robust. Strategy: fix ν, choose M by early stopping (validation curve / CV).

1.9 Boosting-specific hyperparameters

Symbol	Name	Role
M (or B)	# trees / iterations	too small → underfit, too large → overfit; pick via early stopping or CV.
ν	shrinkage / LR	scales each tree; typ. 0.1 or 0.01.
d	tree depth (# splits)	d=1 stumps → additive model; common $4 \le J \le 8$.
J_m	leaves per tree	$= d + 1$; reflects interaction order.
n_{min}	min. obs. per leaf	weak knob given d and M are set.
η_{row}	row subsample fraction	stochastic GBM; typ. 0.5; variance reduction.
η_{col}	column subsample fraction	XGBoost variant.
γ (XGB)	per-leaf penalty	post-fit pruning; larger $\gamma \to$ smaller tree.
λ (XGB)	L2 on leaf weights	ridge-style.
α (XGB)	L1 on leaf weights	lasso-style.
p_{drop} (XGB)	random tree drop	avoid early-tree dominance (Hinton).

1.10 Stochastic gradient boosting

Subsample (without replacement) a fraction η of training rows before fitting each tree (Friedman 2002). Typical $\eta = N/2$. Reduces variance and computation. Variants: subsample rows, subsample columns per tree, subsample columns per split.

1.11 XGBoost (eXtreme Gradient Boosting)

Same forward-stagewise skeleton plus engineering:

- Second-order gradients (Newton): uses gradient + Hessian (2nd-order Taylor) instead of just gradient.
- Parallelisation: split-search across CPU cores.
- Approximate split search: bin features, search bins (not every threshold).
- L1/L2 leaf-weight regularisation + per-leaf pruning γ:

$$\Omega(f) = \gamma|T| + \tfrac{1}{2}\lambda\|w\|^2 \quad \text{(L2)},$$

$$\Omega(f) = \gamma|T| + \alpha\|w\| \quad \text{(L1)}.$$

  $|T|$ = number of leaves, w = leaf-weight vector.
- Row & column subsampling (η_{row}, η_{col}).
- Dropout p_{drop} (Hinton's trick).
- Shrinkage ν as in vanilla GBM.

Internals (Taylor expansion derivation, histogram split search) are out of scope.

1.12 Partial dependence plots (interpretability)

For input X_j, partial dependence of $f(X)$ on X_j:

$$\mathbb{E}_{X_{-j}}\, f(X_j, X_{-j}).$$

Estimated for boosted trees by averaging over the data:

$$\bar{f}(X_j) = \tfrac{1}{N}\sum_{i=1}^{N} f(X_j, (X_{-j})_i).$$

Effect of X_j after accounting for the others; not marginal.

1.13 Boosted-tree squared-error special case

For regression tree + squared-error loss, the boosting subproblem is identical to fitting a single regression tree to the previous-step residuals $r_i = y_i - f(x_i)$ — i.e. gradient boosting = steepest descent on quadratic loss.

1.14 Out-of-scope flags (m09)

Detailed boosting pseudocode line-by-line memorisation; XGBoost / LightGBM / CatBoost internals; package names and code. The conceptual menu is in scope.

2. Derivations

2.1 $\mathrm{Corr}^2(X,Y) = R^2$ in simple LR

Setup. Simple LR $y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$. Shorthand

$$S_{xx} = \textstyle\sum_i (x_i - \bar{x})^2, \qquad S_{yy} = \sum_i (y_i - \bar{y})^2 = \mathrm{TSS},$$
$$S_{xy} = \textstyle\sum_i (x_i - \bar{x})(y_i - \bar{y}).$$

ISLP eq. 3.4: $\hat{\beta}_1 = S_{xy}/S_{xx}$, $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1\bar{x}$. So $\hat{y}_i - \bar{y} = \hat{\beta}_1(x_i - \bar{x})$.

(a) RSS in closed form. Use $y_i - \hat{y}_i = (y_i - \bar{y}) - \hat{\beta}_1(x_i - \bar{x})$:

$$\begin{aligned}
\mathrm{RSS} &= \textstyle\sum_i \left[(y_i - \bar{y}) - \hat{\beta}_1(x_i - \bar{x})\right]^2 \\
&= S_{yy} - 2\hat{\beta}_1 S_{xy} + \hat{\beta}_1^2 S_{xx} \\
&= S_{yy} - 2\,\tfrac{S_{xy}}{S_{xx}}\,S_{xy} + \tfrac{S_{xy}^2}{S_{xx}^2}\,S_{xx} \\
&= S_{yy} - \tfrac{S_{xy}^2}{S_{xx}}.
\end{aligned}$$

(b) Plug into R^2. From ISLP eq. 3.17, $R^2 = 1 - \mathrm{RSS}/\mathrm{TSS}$:

$$R^2 = 1 - \frac{S_{yy} - S_{xy}^2/S_{xx}}{S_{yy}} = \frac{S_{xy}^2}{S_{xx}\,S_{yy}}.$$

(c) Compare to r^2. From ISLP eq. 3.18,

$$r = \mathrm{Corr}(X,Y) = \frac{S_{xy}}{\sqrt{S_{xx}\,S_{yy}}} \;\Rightarrow\; r^2 = \frac{S_{xy}^2}{S_{xx}\,S_{yy}} = R^2. \;\square$$

2.2 ISLP eq. (12.18) and K-means monotonicity

Objective (12.17). Partition $\{1,\dots,n\}$ into $C_1,\dots,C_K$, minimise

$$\sum_{k=1}^{K} W(C_k), \quad W(C_k) = \tfrac{1}{|C_k|}\sum_{i,i' \in C_k}\sum_{j=1}^{p}(x_{ij} - x_{i'j})^2.$$

Identity (12.18). For one cluster C_k with mean $\bar{x}_{kj} = \frac{1}{|C_k|}\sum_{i\in C_k} x_{ij}$:

$$\frac{1}{|C_k|}\sum_{i,i' \in C_k}\sum_{j=1}^{p}(x_{ij} - x_{i'j})^2 \;=\; 2\sum_{i\in C_k}\sum_{j=1}^{p}(x_{ij} - \bar{x}_{kj})^2.$$

Proof (per-feature j; sum then). Fix j; drop the j index. Add/subtract $\bar{x}_k$:

$$\begin{aligned}
\sum_{i,i' \in C_k}(x_i - x_{i'})^2 &= \sum_{i,i' \in C_k}\left[(x_i - \bar{x}_k) - (x_{i'} - \bar{x}_k)\right]^2 \\
&= \sum_{i,i' \in C_k}\Big[(x_i - \bar{x}_k)^2 - 2(x_i - \bar{x}_k)(x_{i'} - \bar{x}_k) \\
&\qquad + (x_{i'} - \bar{x}_k)^2\Big].
\end{aligned}$$

The two squared terms each sum to $|C_k|\sum_{i\in C_k}(x_i - \bar{x}_k)^2$. The cross term factors:

$$\sum_{i,i' \in C_k}(x_i - \bar{x}_k)(x_{i'} - \bar{x}_k) = \Big(\sum_{i\in C_k}(x_i - \bar{x}_k)\Big)^2 = 0$$

because $\sum_{i\in C_k}(x_i - \bar{x}_k) = 0$ by definition of $\bar{x}_k$. So

$$\sum_{i,i' \in C_k}(x_i - x_{i'})^2 = 2\,|C_k|\sum_{i\in C_k}(x_i - \bar{x}_k)^2,$$

divide by $|C_k|$ and sum over j to get (12.18). $\square$

Algorithm decreases objective (per ISLP). Let $Q = 2\sum_k\sum_{i\in C_k}\|x_i - \bar{x}_k\|^2$ (the (12.18) form of the objective). One Lloyd iteration:

2(a) Recompute centroid $\bar{x}_k$ at fixed assignments. The function $c \mapsto \sum_{i \in C_k} \|x_i - c\|^2$ is minimised at $c = \bar{x}_k$ (set derivative to 0: $\sum_{i \in C_k} (x_i - c) = 0 \Rightarrow c = \bar{x}_k$). So Q weakly decreases.

2(b) At fixed centroids, reassign each i to $\arg\min_k \|x_i - \bar{x}_k\|^2$. Each point's contribution weakly decreases, hence Q weakly decreases.

$Q \geq 0$ and only finitely many partitions exist (at most K^n), so the sequence of Q-values must stabilise $\Rightarrow$ convergence to a local optimum. □

2.3 OLS = MLE: univariate and vector

Model. $y = X\beta + \varepsilon$, $\varepsilon \sim N_n(0, \sigma^2 I_n)$. Univariate: $y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$, $\varepsilon_i \overset{iid}{\sim} N(0, \sigma^2)$.

Log-likelihood (both forms).

$$\begin{aligned} \log L(\beta, \sigma^2) &= -\tfrac{n}{2} \log(2\pi\sigma^2) - \tfrac{1}{2\sigma^2} \textstyle\sum_i (y_i - x_i^\top \beta)^2 \\ &= -\tfrac{n}{2} \log(2\pi\sigma^2) - \tfrac{1}{2\sigma^2} \|y - X\beta\|_2^2. \end{aligned}$$

The first term and the $1/(2\sigma^2)$ factor are constant in β, so $\arg\max_\beta \log L = \arg\min_\beta \mathrm{RSS}(\beta)$. Thus $\hat{\beta}_{MLE} = \hat{\beta}_{LS}$. □

Solve the LS problem — side-by-side.

Univariate	Vector
$\mathrm{RSS} = \sum_i (y_i - \beta_0 - \beta_1 x_i)^2$	$\mathrm{RSS} = (y - X\beta)^\top (y - X\beta)$
Expand:	Expand (scalar $y^\top X\beta = \beta^\top X^\top y$):
$\sum y_i^2 - 2\beta_0 \sum y_i - 2\beta_1 \sum x_i y_i + n\beta_0^2 + 2\beta_0\beta_1 \sum x_i + \beta_1^2 \sum x_i^2$	$y^\top y - 2\beta^\top X^\top y + \beta^\top X^\top X \beta$
$\partial/\partial\beta_0 = 0$: $\sum y_i = n\beta_0 + \beta_1 \sum x_i$	$\partial/\partial\beta = -2X^\top y + 2X^\top X\beta = 0$
$\Rightarrow \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$	(uses $\partial a^\top \beta/\partial\beta = a$; $\partial \beta^\top A\beta/\partial\beta = 2A\beta$, A sym.)
$\partial/\partial\beta_1 = 0$ and substitute $\hat{\beta}_0$: $\sum x_i y_i - \bar{y} \sum x_i - \hat{\beta}_1 (\sum x_i^2 - \bar{x} \sum x_i) = 0$	Normal equations: $X^\top X \hat{\beta} = X^\top y$
$\Rightarrow \hat{\beta}_1 = \dfrac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$	If X full column rank: $\boxed{\hat{\beta} = (X^\top X)^{-1} X^\top y}$
$\Rightarrow \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$	2nd deriv. $2X^\top X \succ 0 \Rightarrow$ unique min.

Consistency check. For $X = [1\ x]$, $(X^\top X)^{-1} X^\top y$ reproduces the simple-LR pair above (ISLP eq. 3.4).

2.4 Bias-variance decomposition

Setup. Truth $y_0 = f(x_0) + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$, $\mathrm{Var}(\varepsilon) = \sigma^2$, $\varepsilon \perp \hat{f}$. $\hat{f}$ trained on a random sample; $\mathbb{E}[\cdot]$ is jointly over training set and noise ε.

Step 1 — peel off irreducible noise. Substitute $y_0 = f + \varepsilon$ (suppress x_0):

$$\begin{aligned} \mathbb{E}[(y_0 - \hat{f})^2] &= \mathbb{E}\big[(f - \hat{f} + \varepsilon)^2\big] \\ &= \mathbb{E}[(f - \hat{f})^2] + \mathbb{E}[\varepsilon^2] + 2\,\mathbb{E}[(f - \hat{f})\varepsilon]. \end{aligned}$$

$\mathbb{E}[\varepsilon^2] = \mathrm{Var}(\varepsilon) + \mathbb{E}[\varepsilon]^2 = \sigma^2$. Cross term vanishes: $\varepsilon \perp (f - \hat{f})$ and $\mathbb{E}[\varepsilon] = 0$, so $\mathbb{E}[(f - \hat{f})\varepsilon] = \mathbb{E}[f - \hat{f}]\ \mathbb{E}[\varepsilon] = 0$. Hence

$$\mathbb{E}[(y_0 - \hat{f})^2] = \mathbb{E}[(f - \hat{f})^2] + \sigma^2.$$

Step 2 — add-subtract $\mathbb{E}[\hat{f}]$.

$$f - \hat{f} = \underbrace{(f - \mathbb{E}[\hat{f}])}_{\text{constant in expectation}} + \underbrace{(\mathbb{E}[\hat{f}] - \hat{f})}_{\text{mean-zero}}.$$

Square:

$$\begin{aligned} (f - \hat{f})^2 = (f - \mathbb{E}[\hat{f}])^2 &+ (\mathbb{E}[\hat{f}] - \hat{f})^2 \\ &+ 2(f - \mathbb{E}[\hat{f}])(\mathbb{E}[\hat{f}] - \hat{f}). \end{aligned}$$

Take $\mathbb{E}[\cdot]$. First term constant: $(f - \mathbb{E}[\hat{f}])^2 = \mathrm{Bias}(\hat{f})^2$. Second term: $\mathbb{E}[(\hat{f} - \mathbb{E}[\hat{f}])^2] = \mathrm{Var}(\hat{f})$. Cross term:

$$2(f - \mathbb{E}[\hat{f}]) \cdot \mathbb{E}\big[\mathbb{E}[\hat{f}] - \hat{f}\big] = 2(f - \mathbb{E}[\hat{f}]) \cdot 0 = 0,$$

since $\mathbb{E}[\mathbb{E}[\hat{f}]] = \mathbb{E}[\hat{f}]$.

Combine.

$$\boxed{\mathbb{E}[(y_0 - \hat{f}(x_0))^2] = \mathrm{Var}(\hat{f}(x_0)) + \mathrm{Bias}(\hat{f}(x_0))^2 + \sigma^2.}$$

Reading the terms. σ^2 = irreducible noise floor. $\mathrm{Bias}^2 = (f(x_0) - \mathbb{E}[\hat{f}(x_0)])^2$: systematic error from wrong model class. $\mathrm{Var}(\hat{f}(x_0))$: jitter of the fit at x_0 across resampled training sets. Two cross-term cancellations: (i) $\mathbb{E}[\varepsilon] = 0$; (ii) $\mathbb{E}[\mathbb{E}[\hat{f}] - \hat{f}] = 0$.

3. Vector math

Foundational matrix calculus + matrix-statistics rules. Convention: $\beta, x \in \mathbb{R}^p$ are column vectors; $\partial(\cdot)/\partial\beta$ is a column vector (numerator-layout for scalar-by-vector).

3.1 Matrix derivatives (scalar by vector)

Expression $f(\beta)$	$\partial f/\partial\beta$	Notes
$a^\top\beta = \beta^\top a$	a	Scalar; transpose of a 1×1 is itself.
$\beta^\top A\beta$	$(A + A^\top)\beta$	General square A.
$\beta^\top A\beta$	$2A\beta$	Symmetric $A = A^\top$ (this is the case OLS uses).
$\beta^\top\beta = \|\beta\|^2$	2β	Special case, $A = I$.
$\|X\beta - y\|^2$	$2X^\top X\beta - 2X^\top y$	OLS RSS; sets to $0 \Rightarrow$ normal eqns.

Scalar-by-vector chain rule. For $h(\beta) = g(u(\beta))$ with $u : \mathbb{R}^p \to \mathbb{R}^m$ and $g : \mathbb{R}^m \to \mathbb{R}$:

$$\frac{\partial h}{\partial \beta} = \left(\frac{\partial u}{\partial \beta}\right)^\top \frac{\partial g}{\partial u}.$$

OLS derivation in three lines

$$\begin{aligned} \mathrm{RSS}(\beta) &= (y - X\beta)^\top (y - X\beta) \\ &= y^\top y - 2\beta^\top X^\top y + \beta^\top X^\top X\beta, \\ \tfrac{\partial \mathrm{RSS}}{\partial\beta} &= -2X^\top y + 2X^\top X\beta \overset{!}{=} 0, \\ \hat{\beta} &= (X^\top X)^{-1} X^\top y. \quad \square \end{aligned}$$

The two cross-terms $y^\top X\beta$ and $\beta^\top X^\top y$ collapse because each is a 1×1 scalar and equals its own transpose.

3.2 Expectation, variance, covariance for random vectors

Let $X \in \mathbb{R}^p$ be a random vector, $\mu = \mathbb{E}[X]$, $\Sigma = \mathrm{Cov}(X) = \mathbb{E}[(X-\mu)(X-\mu)^\top]$. A, B, C are constant (non-random) matrices of compatible shape.

Rule	Statement
Sum of expectations	$\mathbb{E}[X+Y] = \mathbb{E}[X] + \mathbb{E}[Y]$
Linear $\mathbb{E}$ of matrix	$\mathbb{E}[AXB] = A\,\mathbb{E}[X]\,B$
Affine $\mathbb{E}$	$\mathbb{E}[AX+b] = A\mu + b$
Σ definition	$\Sigma = \mathbb{E}[XX^\top] - \mu\mu^\top$
Linear transform of cov	$\mathrm{Cov}(CX) = C\,\Sigma\,C^\top$
Affine transform of cov	$\mathrm{Cov}(AX+b) = A\,\Sigma\,A^\top$
Scalar quadratic form	$\mathrm{Var}(a^\top X) = a^\top \Sigma a \geq 0$ (so Σ is PSD)
Cross-covariance	$\mathrm{Cov}(AX, BY) = A\,\mathrm{Cov}(X,Y)\,B^\top$
Univariate sanity	$\mathbb{E}[aX+b] = a\,\mathbb{E}[X] + b$, $\mathrm{Var}(aX+b) = a^2\,\mathrm{Var}(X)$
Sum of two	$\mathrm{Var}(X+Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) + 2\,\mathrm{Cov}(X,Y)$

3.3 Sample covariance matrix from a data matrix

Data matrix $X \in \mathbb{R}^{n\times p}$: rows = observations, columns = variables (ISLP convention; some books transpose). Let $\mathbf{1}_n \in \mathbb{R}^n$ be the all-ones column, $\bar{x} = \frac{1}{n} X^\top \mathbf{1}_n \in \mathbb{R}^p$ the column-means vector.

1. Center (subtract column means from every row):

$$X_c = X - \mathbf{1}_n \bar{x}^\top \in \mathbb{R}^{n\times p}.$$

   Now each column of X_c has mean 0.
2. (For PCA only) Standardize: divide each column of X_c by its sample SD $s_j = \sqrt{\frac{1}{n-1}\sum_i (x_{ij} - \bar{x}_j)^2}$, giving the z-score matrix X_z. Then every column has mean 0, sample variance 1, and $\sum_k \lambda_k = p$.
3. Sample covariance matrix:

$$\boxed{S = \frac{1}{n-1} X_c^\top X_c \in \mathbb{R}^{p\times p}.}$$

   Symmetric, PSD. Entries: $S_{jk} = \frac{1}{n-1}\sum_i (x_{ij} - \bar{x}_j)(x_{ik} - \bar{x}_k)$. Used as Σ in PCA's eigendecomposition.

Connection to OLS algebra: if X already includes the intercept column of ones, then $X^\top X$ (no centering, no $1/(n-1)$) is the un-normalized cross-product used in $\hat{\beta} = (X^\top X)^{-1} X^\top y$ and $\mathrm{Cov}(\hat{\beta}) = \sigma^2 (X^\top X)^{-1}$. Centering kills the off-diagonal between intercept and slopes — numerical reason to center.

4. Misc — Comparison tables

4.1 Table 1 — Hyperparameters & bias-variance direction

Convention: "↑ hp" = increasing the hyperparameter. "B" = bias, "V" = variance. Arrow direction is the effect of increasing the listed hyperparameter, holding others fixed.

Model	Hyperparameter	Meaning	↑ hp effect	Notes / source
OLS	–	no tuning knob	–	special case $\lambda=0$ of ridge
Ridge	$\lambda \geq 0$	L2 penalty strength	B↑, V↓	$\lambda=0$ OLS; $\lambda\to\infty$ all $\hat{\beta}\to 0$
Lasso	$\lambda \geq 0$	L1 penalty strength	B↑, V↓	at finite λ all $\hat{\beta}=0$; sparsifies
Elastic net	λ (overall) $\alpha \in [0,1]$	combined L1+L2 strength L1 mix ($\alpha=1$ lasso)	B↑, V↓ sparsity↑, averaging↓	two knobs; CV over 2D grid lib parametrisation
Subset selection	k (model size)	number of predictors kept	B↓, V↑	best/forward/backward; CV picks k
Logistic regr.	(threshold τ)	decision threshold on $\hat{p}$	not a B-V knob	threshold tunes sens/spec, not flex
LDA	–	no tuning knob	–	shared Σ; τ on posterior is sens/spec
QDA	–	no tuning knob	–	per-class Σ_k; more flex than LDA
Naive Bayes	–	no tuning knob (in course)	–	independence assumption is the "knob"
KNN (clf / regr.)	K	# neighbours averaged/voted	B↑, V↓	K=1 wiggly; K=n const
Polynomial regr.	degree d	order of polynomial fit	B↓, V↑	d+1 params (incl. intercept)
Regression splines	# knots K df (= K+·)	breakpoints in piecewise poly effective complexity	B↓, V↑ B↓, V↑	cubic spline K+4 dof; nat. K dof nat. cubic K=3 knots → 4 dof (2024)
Smoothing splines	$\lambda \geq 0$ df_λ	curvature penalty $\int g''^2$ trace of smoother S_λ	B↑, V↓ B↓, V↑	direction TRAP: opp. of d/K $df_\lambda \to$ n as $\lambda\to 0$
Local regr. (LOESS)	span $s = k/n$	fraction of pts in local fit	B↑, V↓	wide s smooth; narrow s choppy
GAM	df per term	flex of each f_j	B↓, V↑ (per term)	sum dof = total; no interactions
Decision tree	$\alpha \geq 0$ tree size |T| n_{min}	cost-complexity penalty on |T| # terminal leaves min. obs. per leaf	B↑, V↓ B↓, V↑ B↑, V↓	$\alpha=0$ keep full T_0; CV picks α moves opposite to α "lax"; not the selection knob
Bagging	B (# trees)	ensemble size	V↓ then plateau	"use enough", not tuned (OOB)
Random forest	B m	# trees preds per split	V↓ then plateau V↓ (decorrelate); B↑	not a tuning param (use OOB) def. $\sqrt{p}$ clf, $p/3$ regr
AdaBoost	M base depth	# boosting rounds weak-learner size	B↓, V↑ (overfit) B↓, V↑	sequential; can overfit stumps typical
Gradient boost (GBM)	M (# trees) ν d (or J) n_{min}	boosting iterations learning rate / shrinkage tree size, interaction order node-size floor	B↓, V↑ past min V↑ (each step bigger) B↓, V↑ B↑, V↓	tune by CV / early stop $\nu \leq 0.1$; couples with M J∈[4, 8] typical; J=2 stump secondary
Stochastic GBM	η_{row} η_{col}	row subsample fraction column subsample fraction	V↓ (decorrelate) V↓	def. 0.5; without replacement analogue of RF m
XGBoost	M ν d λ α γ η_{row}, η_{col}	# boosting rounds learning rate tree depth L2 on leaf weights L1 on leaf weights per-leaf penalty (prune) row / col subsample	B↓, V↑ past min V↑ B↓, V↑ B↑, V↓ B↑, V↓ (sparser) B↑, V↓ V↓	use early stopping e.g. 0.05–0.1 3 typical ridge on leaves lasso on leaves shrinks |T| SGD-style regulariser
PCR	M (# PCs kept)	dim of reduced regression	B↓, V↑	M<p; CV picks M; unsupervised
PLS	M (# components)	dim of supervised reduction	B↓, V↑	uses Y; behaves like ridge/PCR
Neural net (FNN)	width M depth learning rate weight decay λ dropout p_{drop} epochs (early stop)	hidden units / layer # hidden layers SGD step size L2 (or L1) on weights frac. nodes zeroed training-time cap	B↓, V↑ B↓, V↑ opt. instability if ↑ B↑, V↓ B↑, V↓ B↓, V↑ past min	engineered, no theory for M stacks same recipe implicit regulariser = ridge / lasso on w Hinton; "never train w/o reg." ridge-like effect
K-means	K (# clusters)	partition cardinality	within-SS↓, interp↓	no CV; pick by elbow / use
Hierarchical	linkage cut height	set-to-set dissim. rule induces # clusters	shapes dendrogram –	complete/avg balanced; single chains no K required at fit time

4.2 Table 2 — PCA vs PCR vs PLS

Method	Sup.?	Direction maximises / use case	ISLP
PCA	No	$\mathrm{Var}(X\phi)$, $\|\phi\|=1$. Eigvecs of $\hat{\Sigma}_X$. Unsupervised dim. reduction; viz; decorrelation. Use when compressing X alone (no Y).	§12.2
PCR	No (uses PCA)	$\mathrm{Var}(X\phi)$, then OLS of Y on first M PCs. "Compress, then regress." Tune M by CV. Best when high-variance X-dirs also drive Y.	§6.3.1
PLS	Yes	$\mathrm{Cov}(X\phi, Y)$, $\|\phi\|=1$; deflate & iterate. $\phi_{j1} \propto \mathrm{Corr}(X_j, Y)$. Best when Y depends on low-variance X-dirs (where PCR fails); usually $\approx$ ridge/PCR.	§6.3.2

5. Misc — Notes

1. Which models tolerate p > n?

Model	p > n?	Why
OLS	no	$X^\top X$ singular; infinitely many zero-RSS fits.
Forward stepwise	yes	Builds incrementally; cap at $\mathcal{M}_0, \ldots, \mathcal{M}_{n-1}$.
Backward stepwise	no	Needs full-p OLS as starting point.
Best-subset	no*	Same OLS pathology per subset; * ok for subsets of size < n.
Ridge	yes	$X^\top X + \lambda I$ is PD $\Rightarrow$ unique $\hat{\beta}$ even at p > n.
Lasso	yes	Penalty selects $\leq$ n active features.
Elastic net	yes	Inherits lasso's $\leq$ n active set; ridge piece keeps it unique.
PCR / PLS	yes	Compress to M < min(n, p) components, then OLS.
Logistic regression	no	Same Hessian-singularity story as OLS; needs penalised variant.
LDA	no	Pooled $\hat{\Sigma}$ ($p \times p$) singular when $p \geq n - K$.
QDA	no	Worse: per-class $\hat{\Sigma}_k$ singular when $p \geq n_k - 1$.
Naive Bayes	yes	Diagonal $\Sigma \Rightarrow$ p univariate fits; the $p \gg n$ method.
KNN	yes†	Algorithmically fine; † distances degrade (curse of dim).
Decision tree	yes	Splits one variable at a time; no joint inversion.
Random forest	yes	Each tree picks $m \approx \sqrt{p}$ features; same as tree.
Boosting (trees)	yes	Shallow trees, one split at a time on residuals.
Neural net	yes	Over-parametrised regime is the default (double descent).
GAM (basis)	no‡	Stacked OLS on $X = (1, X_1, \ldots)$; ‡ p > n kills it unless components are penalised.
Smoothing spline	yes	Curvature penalty $\lambda \int g''^2$ regularises the n-knot fit.

2. Degrees of freedom (effective)

Recipe: linear smoother $\hat{y} = Sy \Rightarrow df = \mathrm{tr}(S)$. Wiggly $\uparrow \Leftrightarrow$ df $\uparrow$.

Model	df	Notes
OLS (p predictors)	$p + 1$	Intercept counted; $\mathrm{tr}(H) = p + 1$.
Ridge	$\sum_{j=1}^{p} \frac{d_j^2}{d_j^2 + \lambda}$	d_j = SV of X; $\lambda{=}0 \Rightarrow p$; $\lambda \to \infty \Rightarrow 0$.
Lasso	$\lvert\{j : \hat{\beta}_j \neq 0\}\rvert$	Size of active set (Zou-Hastie).
KNN regression	n/K	ISLP §3.5 framing; large K $\Rightarrow$ low df.
Polynomial deg d	$d + 1$	Intercept + $x, x^2, \ldots, x^d$.
Step function (K cuts)	$K + 1$	K+1 bin-indicators; one absorbed by intercept.
Cubic spline, K knots	$K + 4$	Truncated-power: $\{1, x, x^2, x^3, (x - c_j)_+^3\}$.
Natural cubic, K knots	$K + 2$	Two boundary-linearity constraints; ISLP counts as K excl. intercept.
Smoothing spline	$df_\lambda = \mathrm{tr}(S_\lambda)$	$\lambda{=}0 \Rightarrow n$; $\lambda \to \infty \Rightarrow 2$ (line).
Local regression (LOESS)	tr(S)	Span s=k/n is the knob; linear smoother.
GAM	$\sum_j df_j$	Sum of per-term df (intercept once).
Plain logistic (p pred.)	$p + 1$	Same count as OLS; "free parameters".

3. Standardize? Center? Exhaustive list.

Method	Std.	Center	Req'd?	Why
OLS	no	no	either	Scale/shift-equivariant; std. only for interpreting $\lvert\hat{\beta}_j\rvert$.
Ridge	yes	yes	required	$\sum \beta_j^2$ penalty not scale-invariant; β_0 unpenalised $\Rightarrow$ center y.
Lasso	yes	yes	required	$\sum \lvert\beta_j\rvert$ penalty same; β_0 unpenalised.
Elastic net	yes	yes	required	Both penalties scale-asymmetric.
Logistic regression	no	no	either	Scale-equivariant in β; std. only for numerics/interpretation.
LDA	no	no	either	Discriminant invariant under affine reparametrisation of X.
QDA	no	no	either	Same: per-class Gaussian fit absorbs rescaling.
Naive Bayes	no	no	either	Per-coord 1-D densities; std. only helps SD estimation when scales clash.
KNN (reg/class)	yes	no	required	Euclidean distance dominated by largest-unit coord.
PCA	yes	yes	required	Chases total variance; centering needed so $X^\top X/(n-1) = \hat{\Sigma}$.
PCR	yes	yes	required	Pipeline = standardise $\to$ PCA $\to$ OLS.
PLS	yes	yes	required	Same; uses $\mathrm{Cov}(X, Y)$, scale-sensitive.
Decision tree	no	no	no	Splits compare values within one variable; monotone-invariant.
Random forest	no	no	no	Trees are monotone-invariant; same reason.
Boosting (trees)	no	no	no	Same; XGBoost/LightGBM don't need it.
Neural net (MLP)	yes	yes	required	Imbalanced input scales $\Rightarrow$ imbalanced gradients; bad training.
K-means	yes	no	required	Euclidean distance; nanogram-vs-cm trap.
Hierarchical clust.	yes	no	required	Same Euclidean/linkage distance issue.
Polynomial / step	no	no	either	OLS underneath; scale just rescales β.
Regression splines	no	no	either	OLS on basis-expanded X; scale-equivariant.
Smoothing splines	no	no	either	Penalty is on $\int g''^2$ in x-units; rescaling rescales λ.
GAM	no	no	either	Each f_j fit separately; rescaling X_j rescales f_j.

Rules of thumb. Yes-required = method uses a cross-coordinate penalty or distance (L_2/L_1, variance, Euclidean). Trees are immune: monotone-invariant splits. Compute $\bar{x}_j, s_j$ on the train set only; apply to test. Don't z-score the response y (except for NN training stability), nor 0/1 dummies. Categorical dummies stay 0/1.

4. Sigmoid and logit

Sigmoid (inverse logit):

$$\sigma(z) \;=\; \frac{1}{1+e^{-z}} \;=\; \frac{e^z}{1+e^z}, \qquad \sigma : \mathbb{R} \to (0,1).$$

Logit (inverse sigmoid, "log-odds"): for $p \in (0,1)$,

$$\mathrm{logit}(p) \;=\; \log\frac{p}{1-p} \;=\; \sigma^{-1}(p).$$

Derivative (load-bearing for logreg score & NN backprop):

$$\sigma'(z) \;=\; \sigma(z)\big(1-\sigma(z)\big).$$

Shape. Monotone S-curve. $\sigma(0) = \frac{1}{2}$; $\sigma(-z) = 1 - \sigma(z)$; $\sigma(z) \to 0$ as $z \to -\infty$, $\sigma(z) \to 1$ as $z \to +\infty$; inflection at $z{=}0$ where $\sigma'(0) = \frac{1}{4}$ (max slope).

Logistic regression link. $\log \frac{p_i}{1-p_i} = \beta^\top x_i \;\Leftrightarrow\; p_i = \sigma(\beta^\top x_i)$. Coefficient β_j = log odds-ratio; e^{β_j} = multiplicative effect on odds for a unit increase in x_j.

6. Prewritten answer: decomposition vs. trade-off

Q: Why prefer "bias-variance decomposition" over "bias-variance trade-off"?

A (A-level, tight): "Decomposition" is preferred because the identity $\mathbb{E}[(y_0 - \hat{f}(x_0))^2] = \mathrm{Bias}(\hat{f}(x_0))^2 + \mathrm{Var}(\hat{f}(x_0)) + \mathrm{Var}(\varepsilon)$ is an algebraic equality that holds for every estimator — not a constraint forcing one term up when the other goes down. Two angles make the "trade-off" label misleading: (i) the bias term

is squared, so a small absolute bias contributes little to MSE while variance can shrink a lot (this is exactly why regularisation helps); and (ii) changing the model class — e.g. moving into the over-parameterised / double-descent regime of large NNs — can lower variance without a matching rise in bias. Stating either angle clearly earns the 1 P.

7. Book lookups (ISLP)

ISLP §	What to find there
3.3.3	Potential Problems — linear-model assumption violations and how to gauge them (incl. all the standard diagnostic plots).
3.2.2 (concrete in 3.0)	The questions one might want to ask in linreg (concrete worked version sits at the chapter opener, 3.0).
4.5	Comparison of classification methods — LDA, QDA, Naive Bayes, logistic regression, KNN.

8. Statsbook (Anders)

[Anders: fill in from statsheftet.]